Prerequisites

This tutorial assumes basic familiarity with Kubernetes and `kubectl` but does not assume any pre-existing deployment.
This tutorial assumes that you are familiar with the Terraform workflows. If you are new to Terraform, complete the Get Started collection first.

For this tutorial, you will need the following:

- An Azure account and credentials to use to authenticate.
- The Azure CLI installed locally.
- The `kubectl` CLI installed locally.

Ensure that you are logged into Azure with the `az login` command:

```
$ az login
```

Set up and initialize your Terraform workspace

In your terminal, clone the example repository. This repository contains the example configuration used in this tutorial.

```
$ git clone https://github.com/hashicorp-education/learn-terraform-provision-aks-cluster
git clone https://github.com/VinayKumar1379/Terraform-code-for-EKS-AKS-GKE.git
```

Navigate to the cloned repository.

```
$ cd learn-terraform-provision-aks-cluster or cd Terraform-code-for-EKS-AKS-GKE/Terraform For-aks-cluster
```

Here, you will find the files used to provision the AKS cluster.

1. `aks-cluster.tf` provisions a resource group and an AKS cluster. The `default_node_pool` defines the number of VMs and the VM type the cluster uses.

```
2. resource "azurerm_kubernetes_cluster" "default" {
3.   name                = "${random_pet.prefix.id}-aks"
4.   location            = azurerm_resource_group.default.location
5.   resource_group_name = azurerm_resource_group.default.name
6.   dns_prefix          = "${random_pet.prefix.id}-k8s"
7.   kubernetes_version  = "1.34"
8.
9.   default_node_pool {
10.     name            = "default"
11.     node_count      = 2
12.     vm_size         = "Standard_D2_v4"
13.     os_disk_size_gb = 30
14.   }
15.
16.   service_principal {
17.     client_id     = var.appId
18.     client_secret = var.password
19.   }
20.
21.   role_based_access_control_enabled = true
22.
23.   tags = {
24.     environment = "Demo"
25.   }
26. }
```

27. `variables.tf` declares the `appID` and `password` used to provision the AKS cluster.

Create an Active Directory service principal account

There are many ways to authenticate to the Azure provider. In this tutorial, you will use an Active Directory service principal account. You can learn how to authenticate using a different method here.
First, set the `ARM_SUBSCRIPTION_ID` environment variable to your Azure subscription ID.

```
$ ARM_SUBSCRIPTION_ID=
```

Next, create an Active Directory service principal account using the Azure CLI. You will reference the `appId` and `password` this command returns in a later step.

```
$ az ad sp create-for-rbac --skip-assignment
{
  "appId": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "displayName": "azure-cli-2019-04-11-00-46-05",
  "name": "http://azure-cli-2019-04-11-00-46-05",
  "password": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa",
  "tenant": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa"
}
```

Update your `terraform.tfvars` file

The `azurerm_kubernetes_cluster` resource requires the `appId` and `password` returned when you created the service principal. Create a file named `terraform.tfvars`.

```
$ touch terraform.tfvars
```

Next, add your `appId` and `password` to the `terraform.tfvars` file.

terraform.tfvars

```
appId    = "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa"
password = "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa"
```

Initialize Terraform

Next, initialize your Terraform workspace to download the providers used in your configuration.

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/random from the dependency lock file
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Installing hashicorp/random v3.7.2...
- Installed hashicorp/random v3.7.2 (signed by HashiCorp)
- Installing hashicorp/azurerm v4.53.0...
- Installed hashicorp/azurerm v4.53.0 (signed by HashiCorp)

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Provision the AKS cluster

In your initialized directory, run `terraform apply` and review the planned actions. Your terminal output show what resources Terraform will create.

Note

If you get an error that the VM size of Standard_D2_v4 is not allowed in your subscription, you may have reached a resource limit. Refer to the AKS VM size restrictions and region availability documentation for more information.

```
$ terraform apply
An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  ## ...

Plan: 3 to add, 0 to change, 0 to destroy.

  ## ...
```

Terraform reports that it plans to provision an Azure resource group and an AKS cluster. Respond with `yes` to confirm the apply.

When Terraform completes the apply operation, your terminal prints the outputs defined in `outputs.tf`.

```
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

kubernetes_cluster_name = working-yak-aks
resource_group_name = working-yak-rg
```

Configure kubectl

Now that you have provisioned your AKS cluster, configure `kubectl` to connect to your Kubernetes cluster. Run the following command to retrieve the access credentials for your cluster and automatically configure `kubectl`.

```
$ az aks get-credentials --resource-group $(terraform output -raw resource_group_name) --name
$(terraform output -raw kubernetes_cluster_name)
Merged "light-eagle-aks" as current context in /Users/dos/.kube/config
```

This command uses the `terraform output -raw` command to get the values of the `resource_group_name` and `kubernetes_cluster_name` outputs and pass them to the `az` CLI as arguments.

Verify the cluster

Use `kubectl` commands to verify your cluster configuration.

First, get information about the cluster.

```
$ kubectl cluster-info
Kubernetes control plane is running at https://working-yak-k8s-7imdtit5.hcp.westus2.azmk8s.io:443
CoreDNS is running at https://working-yak-k8s-
7imdtit5.hcp.westus2.azmk8s.io:443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
Metrics-server is running at https://working-yak-k8s-
7imdtit5.hcp.westus2.azmk8s.io:443/api/v1/namespaces/kube-system/services/https:metrics-server:/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Notice that the Kubernetes control plane location address contains the `kubernetes_cluster_name` value from the `terraform apply` output above.

Now verify that both worker nodes are a part of the cluster.

```
$ kubectl get nodes
NAME                                STATUS   ROLES    AGE   VERSION
aks-default-27561840-vmss000000     Ready    <none>   26m   v1.34.0
aks-default-27561840-vmss000001     Ready    <none>   27m   v1.34.0
```

You have verified that you can connect to your cluster using `kubectl` and that both worker nodes are healthy. Your cluster is ready to use.

Clean up your workspace

If you'd like to learn how to manage your AKS cluster using the Terraform Kubernetes Provider, leave your cluster running and continue to the Kubernetes provider tutorial.

When you are done with your AKS cluster, destroy your infrastructure. Run the `terraform destroy` command and confirm with `yes` in your terminal.

```
$ terraform destroy
```